

Machine Learning in Production

Midterm 2, Spring 2026

Claire Le Goues and Christian Kaestner

Name: _____

Andrew ID: _____

Instructions:

- Including this cover sheet and the scenario, your exam should have **8** pages. Make sure you are not missing any pages. *You may detach the last page and recycle it after the exam.*
- All questions in this midterm refer to the scenario on **Page 8**. Answers are graded in the context of the scenario; **generic answers that do not relate to the scenario will not receive full credit.**
- The exam has a maximum score of **57** points. The point value of each problem is indicated. We designed the exam anticipating approximately one minute per point, but you have 80 minutes to complete it.
- **Please write legibly.** We are unlikely to be able to grade your solution if we can't read it.
- Space available is commensurate with expected answer length. We use horizontal lines to suggest stopping points where appropriate. You may exceed these limits if it is clear where to find the rest of your answer. **Do NOT write anything you want graded on the back of pages**, which will not be scanned. However, we strongly recommend writing concise, careful answers; short and specific is better than rambling.
- This is a **closed book exam**; no books or electronics allowed. You may refer to 6 sheets of notes (handwritten or typed, both sides).

Question 1: MLOps [15 points]	2
Question 2: Fairness [12 points]	4
Question 3: Explainability/Transparency [13 points]	5
Question 4: Safety and Security [17 points]	6
Scenario: ApplyAI, An Autonomous Job Search Startup	8

Question 1: MLOps [15 points]

All questions in this exam relate to the scenario on the last page.

(a) [4 points] You want enough provenance on the “*ghost job*” *detector* to trace which code, hyperparameters, and data was used to train a given model version. Which would you use?

- ☐ DVC (Data Version Control) ☐ ROAR + GLaaS ☐ Your own custom scripts

The quality your choice prioritizes:

Key tradeoff (what you gain vs. what you give up):

(b) [4 points] For debugging purposes, you want to be able to access old versions of your “*ghost job*” *detector* model. Which versioning strategy would you recommend *for the model*:

- ☐ Store copies of each model
☐ Store parameter diffs between models
☐ Store model internals in an append-only file and track the offset

The quality your choice prioritizes:

Key tradeoff (what you gain vs. what you give up):

(c) [3 points] *ApplyAI* exposes telemetry with Prometheus from its *ATSFormFiller subagent*. For each of the following metrics, indicate what Prometheus metric type would be appropriate:

- `applications_submitted_total`
(increments each time a job application is successfully submitted)
☐ Counter ☐ Gauge ☐ Timer ☐ Histogram ☐ Event ☐ Summary
- `active_form_sessions`
(recording the number of Playwright browser sessions currently open)
☐ Counter ☐ Gauge ☐ Timer ☐ Histogram ☐ Event ☐ Summary
- `processing_time`
(recording the duration the subagent spent on each request)
☐ Counter ☐ Gauge ☐ Timer ☐ Histogram ☐ Event ☐ Summary

(d) [4 points] As a solo founder short on time, you incur technical debt by launching *ApplyAI* with the *ATSFormFiller subagent* hardcoded to the current HTML layouts of five ATS portals (Workday, Greenhouse, Lever, Taleo, iCIMS). You later realize the portals update their UIs much more regularly than you anticipated, often causing silent failures and disrupting your service.

This technical debt is: ☐ Prudent ☐ Reckless

Explanation:

This technical debt is: ☐ Deliberate ☐ Inadvertent

Explanation:

Question 2: Fairness [12 points]

(a) [4 points] In online evaluation, the *ResumeOptimizer subagent* often makes stronger claims about contributions to past projects for male candidates than female (e.g., "drove"/"owned" vs. "supported"/"helped coordinate"). Lots of experimentation and telemetry make you confident that this does *not* affect hiring outcomes. Still, this may create:

☐ a harm of allocation ☐ a harm of representation ☐ both ☐ neither

Brief explanation:

(b) [4 points] You want *ApplyAI* to help correct for socioeconomic injustices (e.g., candidates with wealthier parents have had access to more extracurriculars and better internships). You tailor *ResumeOptimizer* to correct for these unequal starting positions by providing more support to applicants with fewer initial opportunities (in a form of affirmative action). Assuming that you could collect all necessary data, which *fairness measure(s)* best reflect(s) success at this goal?

☐ anti-classification ☐ group fairness ☐ equalized odds

Short justification:

(c) [4 points] Beyond evaluating and tuning your prompts and models, suggest one *system-level intervention* to improve fairness of your product for applicants with and without wealthy parents.

Question 3: Explainability/Transparency [13 points]

(a) [5 points] The development team wants to understand the *global model behavior* of the “ghost job” detector model, i.e., what patterns the model has learned overall, rather than why it made a specific prediction. You are considering *partial dependence plots* (PDP).

Example of an observation about the model's global behavior a PDP could identify in this scenario:

Basic idea of how PDPs work (rough intuition is sufficient, 2-3 sentences):

(b) [4 points] Which explanation approach best satisfies the needs of an *ApplyAI* user who wants to decide whether to trust the system’s “skip” decisions on postings they care about:

- ☐ SHAP force plot showing feature contributions for each skipped posting
- ☐ Plain-language per-posting rules (e.g., from Anchor, “flagged because: posting open 4+ months, no visible hiring at the company”)
- ☐ Partial dependence plots showing how the model's predictions depend on each feature
- ☐ Global feature-importance chart, evaluated on the training data
- ☐ No explanation — reporting the skip decision alone is sufficient

Justification (considering explanation needs):

(c) [4 points] What explanation approach best satisfies the needs of you, the *ApplyAI* developer, investigating why the ghost-job model often erroneously flags postings from small companies:

- ☐ SHAP force plot showing feature contributions for each skipped posting
- ☐ Plain-language per-posting rules (e.g., from Anchor, "flagged because: posting open 4+ months, no visible hiring at the company")
- ☐ Partial dependence plots showing how the model's predictions depend on each feature
- ☐ Global feature-importance chart, evaluated on the training data
- ☐ No explanation — reporting the skip decision alone is sufficient

Justification (considering explanation needs):

Question 4: Safety and Security [17 points]

(a) [8 points] You consider *TalentFlood* deeply unethical, so you decide to perform an *evasion attack* for your customers on the *TalentFlood model*, which is a custom deep learning classifier (not an LLM). You can run inference with your account, but don't have access to model weights.

Describe in a few words how *evasion attacks* work:

Which security property do you want to undermine primarily with the attack (select one):

- ☐ Confidentiality ☐ Integrity ☐ Availability

What is an example of a defense that *TalentFlood* could implement that would make this attack harder for you:

(b) [3 points] A friend suggests the following way to check that the “ghost job” detector is “robust”: *“Take 10,000 recent postings, add random small perturbations to the numeric features (posting age $\pm$ 3 days, headcount $\pm$ 5%), and re-run prediction. If the classification flips on fewer than 2% of postings, you’re good.”*

This measurement provides *meaningful* evidence of robustness against which of the following?

- ☐ Robustness to *adversarial employers* who deliberately craft posting features to evade the detector.
- ☐ Robustness to *natural distribution shift* as hiring patterns change over the next year.
- ☐ Robustness to *noisy or imprecise feature values*, such as stale LinkedIn headcount numbers or imprecise posting timestamps.
- ☐ Robustness to *out-of-distribution data* like companies very different from those in the training set, such as small startups outside the Fortune 500.
- ☐ None of the above.

(c) [6 points] EmailMCP currently exposes a general `send_email(to, subject, body)` capability. A competitor might try to exfiltrate user data or the system prompt by hiding instructions in a resume uploaded to your service. You want to guarantee that the system *cannot send email (a) to recipients the user has not authorized, or (b) containing content the user has not seen*. Prompt-level instructions are not a guarantee.

Propose one or more changes to the MCP interface or the *ApplyAI* architecture (*not* the prompts) that *guarantee* this property even if the LLM is fully compromised by the injected instructions. Briefly describe each change and the property it enforces:

Scenario: ApplyAI, An Autonomous Job Search Startup

Frustrated with the current tech hiring landscape and after experimenting with a lot of personal Claude-based automation, you decide to build a business (as a solo employee) to automate applying for jobs: *ApplyAI*.

Users upload their resume and enter text responses on target roles, preferences, and dealbreakers. The service then identifies real, relevant job openings; tailors the user's resume and writes a custom cover letter; submits applications through hiring companies' portals; and manages email follow ups. Users get pinged only for interviews and offers.

The system is implemented as an agent, using a frontier LLM as its reasoning core. It has access to tools exposed via MCP or as subagents, including:

- JobBoardMCP queries job board APIs (e.g., Indeed, LinkedIn Jobs, etc.) on a schedule to discover relevant new postings.
- AccountMCP authenticates and modifies user accounts with find, get, and set functions.
- ResumeOptimizer Subagent: rewrites the user's base resume and generates a cover letter tailored to a specific job description.
- ATSFormFiller Subagent uses browser automation (Playwright) to navigate application portals (e.g., Workday, Greenhouse, Lever, Taleo, and iCIMS), filling application/screening forms and uploading documents.
- EmailMCP monitors the user's Gmail inbox for application-related messages, parses them into structured events (confirmation, rejection, interview invite), and can send outreach or follow-up emails.
- NotificationMCP sends push and SMS alerts for high-priority events (interview invitation, offer received).

The only place you train your own model (because prompting didn't work as well) is a "ghost job" detector to identify companies that are unserious about hiring, so your system does not waste resources on their postings. Your model is a random forest classifier trained on 80,000 labeled posts (skewed towards Fortune 500 companies) using engineered features like posting age, company headcount signals from LinkedIn and Glassdoor activity, etc.

You will monitor outcomes for your users to refine your system, including the ghost-job detector. As an additional means to test your system, you have also (under a fake name) bought a subscription to TalentFlood, a widely used resume-screening service.

